



Project Report (Part I)

on

Project Title - A Multi-Tenant SaaS Framework for Online Retail System

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

Submitted by

Nidhay Chavan

Gaurav Gautam

Aryan Goud

Under the Guidance of

Ms. Akshata Raut

Assistant Professor

Department of Computer Engineering

(Academic Year. 2026-27)



TCET THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Approved by AICTE & Govt. of Maharashtra | Affiliated to University of Mumbai (UoM)

- Conferred Autonomous Status by UGC for 10 years w.e.f. A.Y. 2019-20
- Accredited with 'A' Grade by National Assessment & Accreditation Council (NAAC)
- Programmes Accredited by National Board of Accreditation (NBA)
- Empowered Autonomous Status Conferred by University of Mumbai for 10 years w.e.f. A.Y. 2025-26



CERTIFICATE

This is to certify that the project entitled **“VendorHood – A Multitenant SaaS Online Retail System”** is a bonafide work of **Nidhay Chavan (Roll No. 20), Gaurav Gautam (Roll No. 44), Aryan Goud (Roll No. 48)** submitted to the Thakur College of Engineering and Technology, Mumbai (An Autonomous College affiliated to University of Mumbai) in partial fulfillment of the requirement for the **Project-I** for award of the degree of **“Bachelor of Engineering”** in **“Computer Engineering”**.

Signature with Date: -----

Name of Guide:

Designation:

Signature with Date: -----

Name of HOD:

Name of Department:



TCET | THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Approved by AICTE & Govt. of Maharashtra | Affiliated to University of Mumbai (UoM)

- Conferred Autonomous Status by UGC for 10 years w.e.f. A.Y. 2019-20
- Accredited with 'A' Grade by National Assessment & Accreditation Council (NAAC)
- Programmes Accredited by National Board of Accreditation (NBA)
- Empowered Autonomous Status Conferred by University of Mumbai for 10 years w.e.f. A.Y. 2025-26



PROJECT APPROVAL CERTIFICATE

This project report entitled “**VendorHood – A Multitenant SaaS Online Retail System**” by *Nidhay Chavan (Roll No. 20)*, *Gaurav Gautam (Roll No. 44)*, *Aryan Goud (Roll No. 48)* is approved for the degree of “**Bachelor of Engineering**” in “**Computer Engineering**”.

Internal Examiner:

External Examiner:

Signature: -----

Signature:

Name:

Name:

Date:

Place:

ACKNOWLEDGEMENT

It would be unfair if I do not acknowledge the help and support given by Professors, students, friends etc.

We sincerely thank our guide Ms. Akshata Raut for her guidance and constant support and also for the stick to our backs. We also thank the project coordinators for arranging the necessary facilities to carry out the project work.

We thank the HOD, Dr. Vaishali Kaiche, the Principal, Dr. B. K. Mishra and the college management for their support.

1. Nidhay Chavan (Roll No. 20)
2. Gaurav Gautam (Roll No. 44)
3. Aryan Goud (Roll No. 48)

Blue Book Plagiarism Report (From
department Turnitin account only)

INDEX

Chapter No.		Page No.
	List of Figures	I
	List of Tables	II
	Abstract	III
Chapter 1	Introduction	
	1.1 Overview of the Project	
	1.2 Motivation & Application	
	1.3 Problem Definition	
	1.4 Objective & Scope	
	1.5 Expected outcome	
	1.6 Organization of the Report	
Chapter 2	Literature Survey & Proposed System	
	2.1 Literature review of Existing System	
	2.2 Limitations of Existing System & Gap Analysis	
	2.3 Proposed System	
Chapter 3	Requirement Gathering, Analysis and Planning	
	3.1 Requirement Specification	
	3.2 Feasibility Study	
	3.3 Methodology	
	3.4 Technology Stack	
	3.5 Gantt Chart and Process Model	
	3.6 System Analysis (Functional, Structural, and Behavioral Models)	
Chapter 4	System Design and Experimental Set up	
	4.1 System Architecture & Diagrams (DFD/UML/ Block Diagram/Physical Layout)	
	4.2 Algorithm & Process flow design (Flowchart/Pseudo Code)	
	4.3 User Interface & Input Data Design (Snapshots/ circuits/Structure)	
	4.4 Experimental Setup and Tools (Software & Hardware)	
	4.5 Implementation, Deployment and Testing	
	4.6 Performance Evaluation.	
	4.7 Summary	
Chapter 5	Results & Discussion	
	5.1 Outputs & Outcomes	
	5.2 Analysis of Results & Interpretation of data	
	5.3 Discussion of Results & Limitations of the System	
Chapter 6	Conclusion & Future Scope	
	6.1 Summary of work completed	
	6.2 Future Scope	

References

Research Paper

Appendix A

- ● Abbreviation and symbols

Appendix B

- • Definitions

Appendix C

- List of Publications

List of Figures

Figure 1: Product Entity Structure	15
Figure 2: Tenant-Specific Storefront Generation Process	15
Figure 3: Commission-Based Payment Splitting and Settlement Flow	16
Figure 4: Overall Proposed Multi-Tenant E-Commerce System Flow	17
Figure 3.1: Functional Relationship	27
Figure 3.2: Tenant Isolation Structure	28
Figure 3.3: Behavioral Model	29
Figure 3.4: Order & Commission Processing	30
Figure 3.5: Storefront Template	30
Figure 4.1: System architecture / physical layout block diagram of VendorHood	32
Figure 4.2: Flowchart: authentication and role-based tenant resolution	33
Figure 4.3: Flowchart: multi-tenant checkout and commission-split algorithm	35
Figure 4.4: VendorHood Landing Page	36
Figure 4.5: Tenant management List for VendorHood Admins	37
Figure 4.6: Platform Analytics for VendorHood Admins	37
Figure 4.7: Product Catalog / Inventory for Vendors	37
Figure 4.8: Shop Analytics + Order Ledger for VendorHood Admins	38
Figure 4.9: Vendor Customizable Storefront	38

List of Tables

Table 1: Literature Review Findings on Multi-Tenant E-Commerce Platforms	13
Table 3.1: Technology Stack	24
Table 4.1: Input data design summary	39
Table 4.2: Software Requirements	39
Table 4.3: Hardware Requirements	40
Table 4.4: Implementation, Deployment and Testing	40

Chapter 1 — Introduction

1.1 Overview of the Project

The project proposes a multi-tenant SaaS-based online retail platform that enables multiple independent vendors to operate their own online storefronts using a common platform infrastructure.

In a conventional e-commerce system, each business may need to invest separately in software development, hosting, customization, maintenance, and operational infrastructure. This creates a high fixed-cost burden, particularly for small businesses and startups.

The proposed system addresses this problem by using multi-tenancy, where multiple vendors share the same application and infrastructure while their data and configurations remain logically separated.

The proposed system consists of several major architectural components, including shared-schema multi-tenancy, tenant identification and tenant-context enforcement, template-based storefront customization, subdomain and custom-domain support, commission-based transaction processing, payment splitting between the platform and vendors, and vendor-managed order fulfillment. The system also considers important challenges such as performance isolation and the noisy-neighbor problem. Overall, the objective is to provide a cost-aligned SaaS retail platform that enables vendors to participate in e-commerce without bearing the complete fixed cost of developing and maintaining an individual e-commerce system.

1.2 Motivation & Application

Motivation

The primary motivation of the project is the high cost associated with entering the digital retail market, as small and micro-scale businesses may face expenses related to software development, software customization, infrastructure, integration, maintenance, training, and continuous operational costs. These expenses may have to be paid before a business generates sufficient online sales, creating an economic barrier to digitalization for businesses with limited capital. Multi-tenancy provides an alternative by allowing several vendors to use a shared application and infrastructure while maintaining logical data isolation, thereby distributing the cost of the underlying system across multiple tenants. The commission-based model further supports this objective because the platform's revenue is linked to transactions rather than being dependent entirely on fixed subscription fees or upfront charges.

Applications

The proposed framework can be applied to several types of businesses and e-commerce models. It is suitable for small online retailers that want to establish an online storefront

without developing an independent system. It can also support multiple-brand marketplaces, where different brands operate through the same SaaS infrastructure. In vendor-based e-commerce platforms, each vendor can have its own tenant and storefront, while startup-oriented retail platforms can use the framework to enter digital commerce with lower initial infrastructure requirements. Additionally, the framework supports custom-domain storefronts, allowing vendors to access and operate their stores through dedicated subdomains or their own custom domains.

1.3 Problem Definition

Small and micro-scale businesses face significant technical and financial barriers when establishing independent online retail systems.

Traditional e-commerce platforms can involve substantial initial and recurring costs, including development, customization, infrastructure, maintenance, integration, and operational expenses. These fixed costs can become a burden before a vendor has generated sufficient revenue.

At the same time, a shared multi-tenant architecture introduces technical challenges. Multiple vendors use the same infrastructure, so the system must prevent one tenant from accessing another tenant's data.

The problem can therefore be defined as:

To design a cost-efficient multi-tenant SaaS framework for online retail that allows multiple vendors to operate independent storefronts on shared infrastructure while maintaining tenant data isolation, customizable storefronts, domain-based tenant identification, and transaction-based commission settlement.

The paper particularly identifies the risk of cross-tenant data leakage in shared-schema architectures and therefore emphasizes tenant-context enforcement throughout.

1.4 Objective & Scope

Objectives

The main objectives of the proposed system are to develop a multi-tenant SaaS architecture that allows multiple vendors to use a common application and infrastructure while maintaining logical separation of their data. The system aims to reduce fixed costs by using shared infrastructure and a common codebase, thereby minimizing development, maintenance, and operational overhead. It also focuses on implementing tenant isolation by ensuring that each request is associated with the correct tenant and that database operations are restricted to that tenant. Another objective is to provide customizable storefronts, allowing vendors to select predefined storefront designs and layouts without requiring custom development for each vendor. The system also aims to implement domain-based tenant resolution by supporting both subdomain-based and custom-domain storefronts. In addition, commission-based monetization will be implemented to process

payments and ensure that the platform and vendor receive their respective shares of each transaction. Finally, the system aims to maintain scalability and performance by considering resource sharing, performance interference, and the noisy-neighbor problem in a shared Java-based environment. These objectives correspond to the framework's four principal design areas: data architecture, presentation layer, domain-specific request routing, and settlement logic.

Scope

The scope of the proposed system includes vendor onboarding, tenant creation, tenant ID generation, domain assignment, customer authentication, tenant-specific product catalogues, storefront generation, product browsing, stock checking, checkout, commission-based payment splitting, order-history updates, tenant data isolation, and subdomain/custom-domain resolution. The system is designed to support the core operations of a multi-tenant e-commerce platform while ensuring proper separation of vendor data. However, the proposed system does not focus on logistics or physical order fulfilment. According to the proposed framework, shipment and fulfilment remain the responsibility of the individual vendors.

1.5 Expected Outcome

The expected outcome of the proposed system is a working architectural framework for a multi-tenant online retail SaaS platform that can support multiple vendors using shared infrastructure. The system is expected to provide reduced infrastructure and operational costs through resource sharing, logical separation of data between tenants, tenant-aware product and catalogue retrieval, customizable storefronts through predefined templates, subdomain and custom-domain support, transaction-based commission processing, split settlement between vendors and the platform, and consistent order-status updates following successful payment. It will also provide a foundation for future scalability and performance isolation. The expected operational flow of the system is vendor onboarding → tenant/domain provisioning → customer login → tenant-specific catalogue browsing → stock checking → checkout → payment split → order update → confirmation.

1.6 Organization of the Report

The report is organized into multiple chapters to present the development and analysis of the proposed multi-tenant SaaS framework systematically. Chapter 1 introduces the project, including its background, motivation, problem definition, objectives, scope, and expected outcomes. Chapter 2 presents the literature survey, analysis of existing systems, identified limitations and research gaps, followed by the proposed system. The subsequent chapters describe the system requirements, architecture and design, implementation methodology, testing and results, followed by the conclusion and future scope of the proposed system.

Chapter 2 — Literature Survey & Proposed System

2.1 Literature Review of Existing System

The literature review examines existing research related to multi-tenant SaaS, e-commerce platforms, tenant isolation, storefront customization, cloud deployment, and performance management.

The reviewed literature establishes several important points:

Table 1: Literature Review Findings on Multi-Tenant E-Commerce Platforms

Area	Finding from Literature
Multi-tenant e-commerce	Sharing application resources can reduce development and maintenance costs
SaaS architecture	Different database/schema isolation strategies can be used for scalable SaaS
Multi-vendor platforms	SaaS platforms can combine storefront creation with additional business functionality
Security	Tenant isolation is a major requirement in multi-tenant systems
Dynamic tenancy	Tenant management can be applied to microservice-based SaaS
Java performance	Shared Java environments can experience cross-tenant performance interference
E-commerce SaaS	Usage/transaction-based charging can improve affordability for smaller businesses
Cloud platforms	Multi-tenant PaaS can provide shared APIs and cloud workflows
SaaS architecture studies	Customization and migration are important challenges
Resource management	Tenant-specific CPU/memory control can help prevent interference
Deployment	Isolation levels involve trade-offs between resources, cost and performance
Microservices	Multi-tenant architectures can improve scalability and reliability

The literature review therefore provides the foundation for combining tenant isolation, storefront customization, domain resolution and commission settlement into one proposed retail framework.

2.2 Limitations of Existing System & Gap Analysis

The research paper identifies several gaps across the reviewed literature. A major gap is the lack of an integrated retail architecture, as several studies focus on general SaaS or multi-tenancy but do not combine all the requirements of an online retail platform. There is also limited commission-based payment integration, as some existing architectures discuss multi-tenancy without specifically incorporating commission-based payment and settlement mechanisms. Storefront customization is another limitation, with many studies focusing primarily on backend architecture and security while giving less attention to tenant-specific frontend and storefront customization. Furthermore, domain resolution is not comprehensively integrated, as existing work may discuss subdomains but does not necessarily combine subdomain and custom-domain resolution with tenant identification and data isolation. Another gap is the limited integration of tenancy and transaction processing, where some frameworks focus on tenant management without incorporating the complete retail transaction flow, including payment splitting and order updates. Finally, performance isolation remains a challenge, as shared Java environments can experience cross-tenant resource interference, particularly when tenants compete for CPU and memory resources.

Research Gap

The central gap addressed by the proposed work is the lack of a framework that combines: Multi-tenancy + tenant isolation + customizable storefronts + domain resolution + commission-based payment settlement within a single online retail SaaS architecture.

This is essentially the contribution your paper is positioning: integrating these components into one framework targeted toward cost-sensitive small and micro-scale retailers.

2.3 Proposed System

The proposed system is a multi-tenant SaaS-based online retail framework where multiple vendors operate their individual storefronts through a common platform.

The system consists of four major architectural areas:

1. Data Architecture

The system uses a shared-schema multi-tenant model.

Tenant-specific records are stored in common tables and identified using a tenant ID/discriminator.

For example:

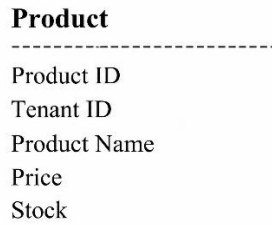


Fig 1: Product Entity Structure

The Tenant ID ensures that a product belongs to a particular vendor. Every request first establishes its tenant context, and database operations are then restricted to that tenant.

2. Presentation Layer

The system uses a template-injection front-end engine. Instead of developing a completely different frontend for every vendor:

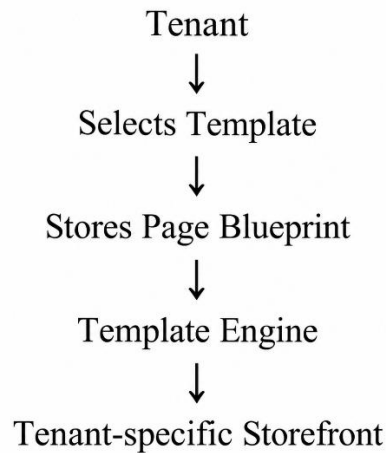


Fig 2: Tenant-Specific Storefront Generation Process

The paper proposes a limited set of predefined designs, generally around 4–5 designs, from which a tenant can select. Tenant-specific content is then inserted into the selected page structure at runtime.

3. Domain & Tenant Resolution

The system supports:

Subdomain:

brand1.platform.com

brand2.platform.com

and

Custom domain:

www.brand1.com

www.brand2.com

The system examines the incoming Host header. If it is a platform subdomain, the tenant key is extracted from the subdomain. Otherwise, the domain is searched in a domain-alias registry to identify the appropriate tenant.

4. Commission-Based Payment

At checkout, the payment is divided between the platform and vendor.

Conceptually:

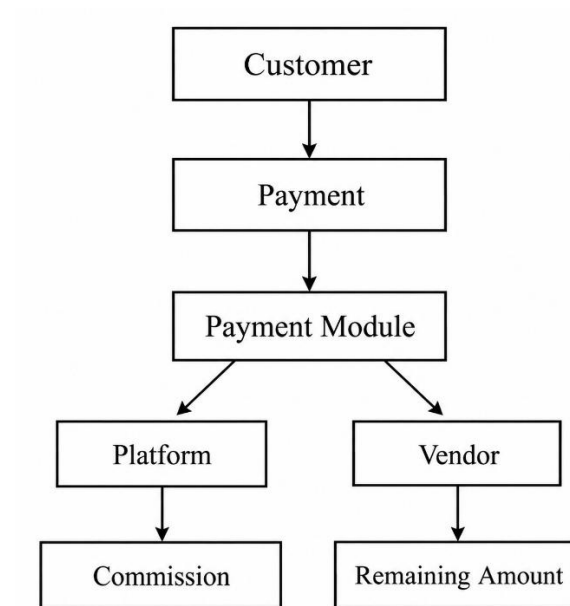


Fig 3: Commission-Based Payment Splitting and Settlement Flow

The paper describes the platform's share as:

Platform Share = Commission $\times$ Transaction Amount

and the vendor's share as:

Vendor Share = (1 – Commission) $\times$ Transaction Amount

The payment gateway's callback/webhook then updates the order status after successful settlement.

Overall Proposed System Flow

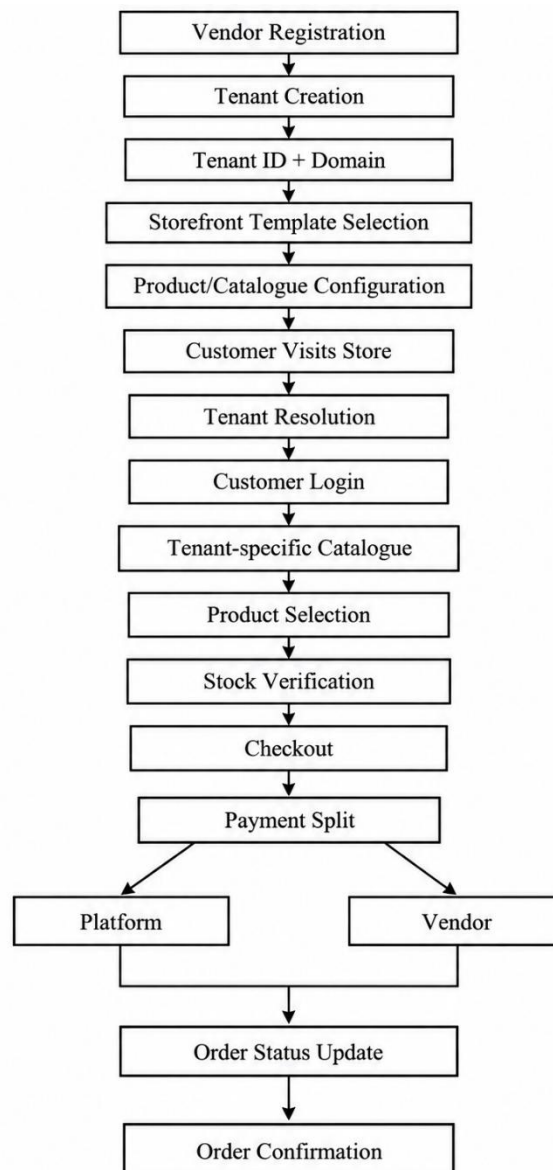


Fig 4: Overall Proposed Multi-Tenant E-Commerce System Flow

VendorHood follows a three-tier web architecture built around a single shared codebase that serves every tenant

CHAPTER 3 - Requirement Gathering, Analysis and Planning

3.1 Requirement Specification

The proposed Vendor Hood system is a multi-tenant Software-as-a-Service (SaaS) platform designed for online retail vendors. The system provides a common application infrastructure through which multiple vendors can operate independent online storefronts while maintaining logical separation of their products, orders, configurations, and transaction information.

The requirements were derived from the objectives of the proposed multi-tenant architecture, the commission-based business model described in the research work, and the implemented Vendor Hood prototype. The major requirements concern tenant management, product management, storefront generation, domain-based tenant resolution, order processing, commission calculation, and administrative monitoring.

3.1.1 Functional Requirements

The functional requirements of VendorHood are presented below.

A. Vendor/Tenant Management

The system shall allow creation of vendor/tenant records, maintain a unique tenant identifier for each vendor, store vendor name and profile information, assign a subdomain to each vendor, support an optional custom domain for a vendor, associate a storefront template with each vendor, store a configurable commission rate for each vendor, allow administrators to view all registered vendors, allow administrators to update vendor information, and allow administrators to delete a vendor and its associated products and orders.

The implemented Vendor entity contains fields including id, name, subdomain, customDomain, templateId, commissionRate, and logoUrl.

B. Product Management

The system shall allow vendors to add, update, and delete products, storing product name, price, description, image, and stock, along with additional attributes for different storefront categories. It displays only products belonging to the active tenant, while allowing administrators to view products across tenants where global administration is required.

The implementation uses a Product entity with a tenant identifier and product-specific attributes. Vendor-level product operations are exposed through the vendor controller, while global product access is provided through administrative operations.

C. Storefront Management

The system shall generate a storefront according to the selected tenant, displaying tenant-specific products with a tenant-selected visual template. It supports multiple predefined

storefront layouts and allows vendors to change their storefront template, while displaying vendor branding such as name and logo. The storefront can be accessed through both subdomain-based and custom-domain-based routes.

The frontend currently contains five predefined storefront templates: Studio Ivory Serif, Stark Dark Mono, Warm Editorial Sand, Stark Swiss Grid, and Heritage Botanical.

The selected `templateId` is stored as part of the vendor configuration and is used by the frontend rendering logic.

D. Domain and Tenant Resolution

The system shall identify the active tenant from the incoming request.

The tenant-resolution process first checks for a tenant identifier, then checks the simulated host/domain information, followed by the actual Host header. It determines whether the host corresponds to a VendorHood subdomain and searches the vendor database using the subdomain; if no subdomain is found, it searches instead for a matching custom domain. The resulting tenant identifier is then assigned to the request context and used throughout subsequent database operations.

E. Order Management

The system shall allow customers to submit orders through a storefront, associating each order with the active tenant and recording the customer information, total order amount, order status, and order date. It calculates the platform commission and the vendor's share for each order, allows vendors to view and update the status of their orders, and allows customers to retrieve orders associated with the relevant storefront.

The Order entity contains fields for tenant ID, customer name, total amount, platform commission, vendor share, status, and date.

F. Commission Calculation

The proposed business model uses transaction-based commission rather than a fixed subscription fee.

For an order with total value P and commission rate r , the platform commission is:

$$C = P \times \frac{r}{100}$$

The vendor share is:

$$V = P - C$$

where P = total transaction amount, r = vendor-specific commission percentage, C = platform commission, and V = vendor share.

The implementation performs this calculation in the storefront order-processing controller. For example, if:

$$P = 10,000$$

and

$$r = 8\%$$

then:

$$C = 10,000 \times 0.08 = 800$$

and:

$$V = 10,000 - 800 = 9,200$$

G. Administrative Management

The platform administrator shall be able to view, create, update, and delete registered vendors, view products and orders across tenants, and monitor total vendor count, total product count, total sales, and total platform commission.

The administrative functionality is implemented through AdminController.

3.1.2 Non-Functional Requirements

A. Multi-Tenant Isolation

Tenant isolation is a primary requirement of VendorHood.

Each tenant must access only its own products, orders, storefront information, and tenant-specific configuration.

B. Scalability

The architecture should support multiple vendors using a common application and database infrastructure. Shared infrastructure reduces duplication and allows additional tenants to be introduced without deploying a completely separate application instance for each vendor.

C. Performance

The system should provide acceptable response times when retrieving storefront information, products, orders, and vendor information.

The research paper additionally identifies the noisy-neighbor problem as a concern in shared multi-tenant environments, particularly where tenants compete for common runtime resources.

D. Security

The system should prevent unauthorized cross-tenant access. Tenant identification must therefore occur before tenant-specific business and database operations.

E. Maintainability

The system should use modular components so that tenant management, product management, order processing, storefront rendering, and administrative functionality can be modified independently.

F. Usability

The system should provide separate interfaces for the platform administrator, vendor, and customer.

The vendor interface provides inventory, order, profile, and storefront-management functionality.

3.2 Feasibility Study

The feasibility study determines whether the proposed VendorHood platform can be developed using the available technical resources and architecture.

3.2.1 Technical Feasibility

The project is technically feasible because the implementation uses established technologies for web application development.

The backend is implemented using Java 17, Spring Boot 3.3.2, Spring Web, Spring Data JPA, Hibernate, and MySQL Connector/J.

The frontend uses React 18, ReactDOM, JSX/Babel, Tailwind CSS, HTML5, CSS3, and JavaScript.

The database is MySQL 8.0 and is configured through Docker Compose.

The architecture also provides mechanisms for tenant context resolution and shared-schema tenant isolation, making the selected technology stack technically suitable for the proposed SaaS model.

3.2.2 Economic Feasibility

The proposed SaaS model is intended to reduce the fixed infrastructure burden on individual vendors by allowing multiple vendors to use shared application infrastructure.

Instead of requiring every vendor to independently maintain application infrastructure, database infrastructure, storefront software, development resources, and maintenance infrastructure, the SaaS platform provides a common platform that can serve multiple vendors.

The commission-only business model also links platform revenue to vendor transactions rather than requiring a fixed subscription payment.

The research paper identifies this cost alignment as one of the primary motivations for the proposed architecture.

Note: The supplied project files do not provide an actual production cost analysis, hosting bill, or payment-provider pricing. Therefore, exact economic savings should not be claimed in this chapter.

3.2.3 Operational Feasibility

The system is operationally feasible because its major user activities correspond to common online-retail workflows.

Platform Administrator

The administrator manages vendors, products, orders, and platform statistics.

Vendor

The vendor manages the store profile, products, inventory, storefront template, orders, and order status.

Customer

The customer interacts with the storefront to browse products, select products, place orders, and view order-related information.

The research architecture deliberately keeps physical logistics outside the platform's core scope, leaving order fulfilment and shipping responsibilities with the vendor.

3.2.4 Schedule Feasibility

The development can be divided into sequential stages, beginning with a literature review and requirement analysis, followed by architecture design and database design. Backend development and frontend development follow, after which multi-tenancy and commission-calculation features are implemented. The final stages consist of integration, testing, performance evaluation, and documentation.

The actual project dates should be entered according to the team's real development timeline.

3.2.5 Security and Data Feasibility

The shared-schema architecture introduces a specific requirement: every tenant-specific database operation must be executed within the correct tenant context.

The implementation addresses this by resolving each incoming request through a `TenantFilter`, which establishes the `TenantContext` using a `TenantIdentifierResolver`; Hibernate then uses this context to access tenant-specific data.

This provides a centralized mechanism for tenant identification and database isolation. The research paper similarly identifies incorrect tenant scoping as a major risk in shared-schema multi-tenancy.

3.3 Methodology

The development methodology adopted for VendorHood is an iterative and modular development approach. This methodology is selected because VendorHood is a multi-tenant SaaS retail platform consisting of several interconnected functional components, including tenant management, vendor management, product and inventory management, customer management, order processing, payment processing, commission calculation, analytics, and administrative operations. Developing the entire system as a single unit would increase complexity and make it difficult to identify and resolve errors during development. Therefore, the system is decomposed into smaller and manageable modules that can be developed, tested, evaluated, and refined independently before being integrated into the complete platform.

3.3.1 Requirement Gathering

Requirements were identified from the problem of high fixed costs associated with conventional online-retail systems and from the architectural requirements of multi-tenant SaaS platforms.

The principal requirements identified were shared infrastructure, tenant isolation, vendor-specific storefronts, product management, order management, commission calculation, domain-based tenant identification, and administrative monitoring.

The research literature further identifies customization, tenant isolation, scalability, and resource utilization as important multi-tenant SaaS concerns.

3.3.2 System Design

The system design is based on four primary architectural components: shared-schema multi-tenancy, template-based storefront generation, domain-based tenant resolution, and commission calculation.

These components collectively form the core of VendorHood.

3.3.3 Implementation

The backend is implemented as a Spring Boot application. JPA/Hibernate is used for persistence, while MySQL provides the database layer.

The frontend provides the administration console, vendor portal, storefront interfaces, and template-based presentation layer.

3.3.4 Testing Strategy

Testing should be performed at multiple levels:

Unit Testing

Testing individual backend components and calculations.

Integration Testing

Testing communication between the frontend, backend, and database.

Tenant Isolation Testing

Testing whether a request for Tenant A can incorrectly retrieve data belonging to Tenant B.

Functional Testing

Functional testing covers vendor creation, product creation, product update, product deletion, order creation, commission calculation, domain resolution, and template selection.

Performance Testing

Testing system behavior as the number of tenants, products, and orders increases.

3.4 Technology Stack

The following technologies are used in the VendorHood implementation.

Table 3.1: Technology Stack

Layer	Technology	Purpose
Frontend	React 18	Component-based user interface
UI Framework	Tailwind CSS	Responsive interface styling
JavaScript/JSX	JavaScript + JSX	Frontend application logic
JSX Compiler	Babel Standalone	Runtime JSX compilation in the supplied frontend
Backend	Java 17	Server-side application
Backend Framework	Spring Boot 3.3.2	REST API and application framework
Web Layer	Spring Web / Spring MVC	HTTP request and REST API handling
Persistence	Spring Data JPA	Database persistence
ORM	Hibernate	Object-relational mapping and tenant handling
Database	MySQL 8.0	Persistent data storage
Database Driver	MySQL Connector/J	Java-MySQL connectivity
Containerization	Docker Compose	Local MySQL deployment
Build Tool	Maven	Backend dependency/build management
Code Simplification	Lombok	Reducing Java boilerplate
Styling	CSS3	Custom animations and UI styles

The backend pom.xml specifies Java 17, Spring Boot 3.3.2, Spring Web, Spring Data JPA, MySQL Connector/J, Lombok, and Spring Boot testing dependencies. The frontend index.html loads React 18, ReactDOM, Babel, and Tailwind CSS through CDNs.

3.4.1 Database Technology

MySQL 8.0 is used as the database management system.

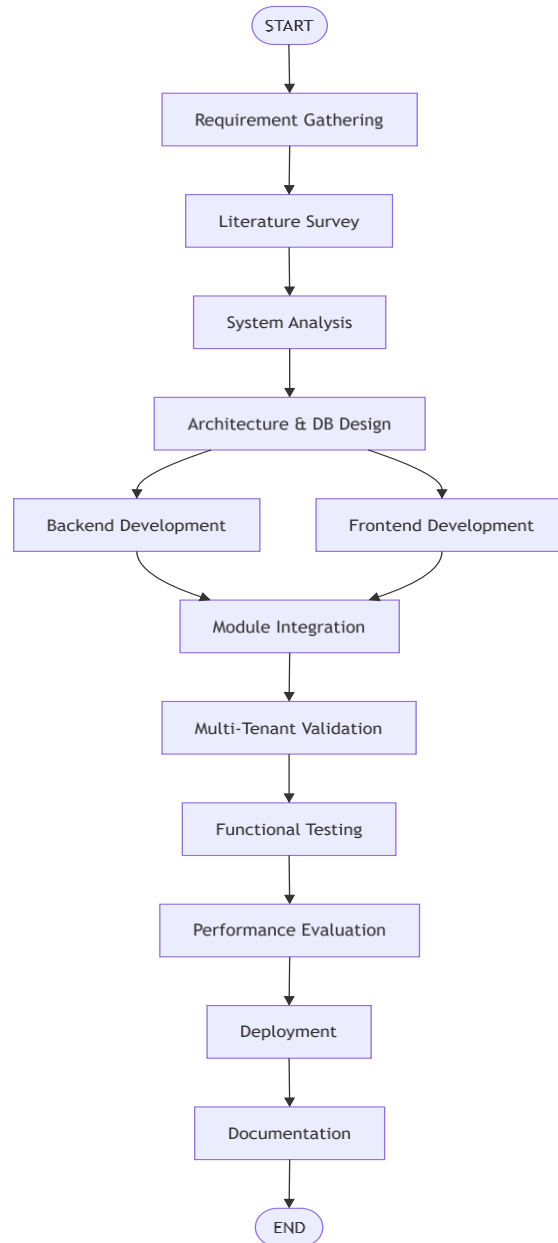
The Docker configuration creates the vendorhood database and exposes MySQL through port 3307 on the host.

The major persistent entities include Vendor, Product, and Order.

Products and orders contain tenant identifiers to support multi-tenant isolation.

3.5 Process Model

3.5.1 Development Process Model



3.5.2 Major Development Activities

Phase Major Activities

Phase 1 Literature survey and problem identification

Phase 2 Requirement gathering and analysis

Phase Major Activities

Phase 3 Multi-tenant architecture design

Phase 4 Database and entity design

Phase 5 Backend API development

Phase 6 Frontend and storefront development

Phase 7 Tenant isolation implementation

Phase 8 Commission calculation implementation

Phase 9 Domain resolution and template integration

Phase 10 Integration and testing

Phase 11 Performance evaluation

Phase 12 Documentation and final deployment

3.6 System Analysis

The VendorHood system is analyzed using three complementary models: a functional model, a structural model, and a behavioral model.

3.6.1 Functional Model

The functional model describes the major operations performed by the system and the interaction between users and the platform.

Major Actors

The primary actors are the platform administrator, the vendor/tenant, and the customer.

Platform Administrator Functions

The administrator can manage, view, update, and delete vendors, view products and orders, and monitor sales and platform commission.

Vendor Functions

The vendor can view, add, update, and delete products, manage the vendor profile, select a storefront template, view and update orders and order status, and view vendor-specific revenue information.

Customer Functions

The customer interacts with the storefront to access a tenant storefront, browse products, place orders, and view customer orders.

Use Case Representation

The functional relationship can be represented as:

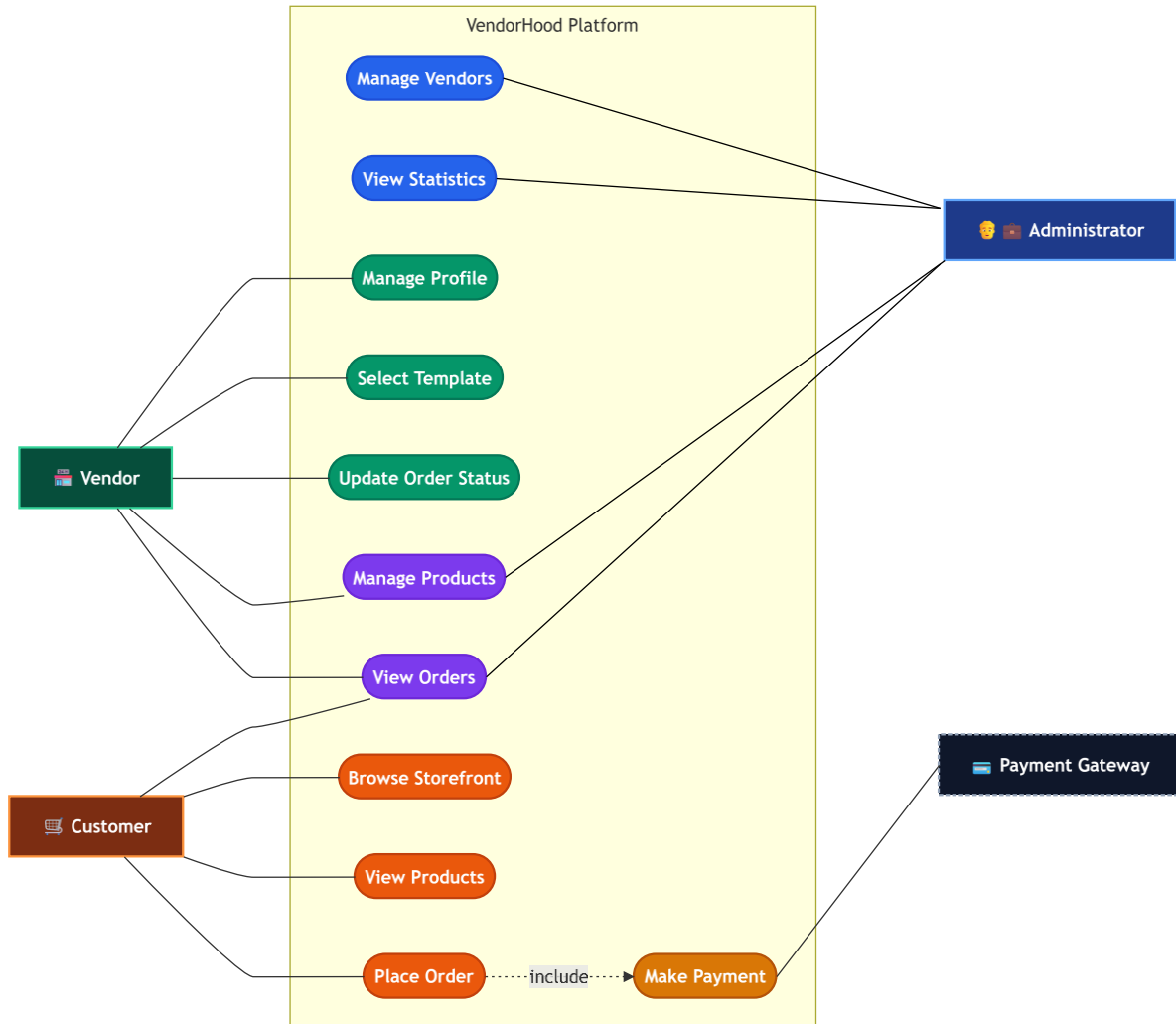


Fig 3.1: Functional Relationship

3.6.2 Tenant Isolation Structure

The tenant isolation architecture is one of the most important structural characteristics of VendorHood.

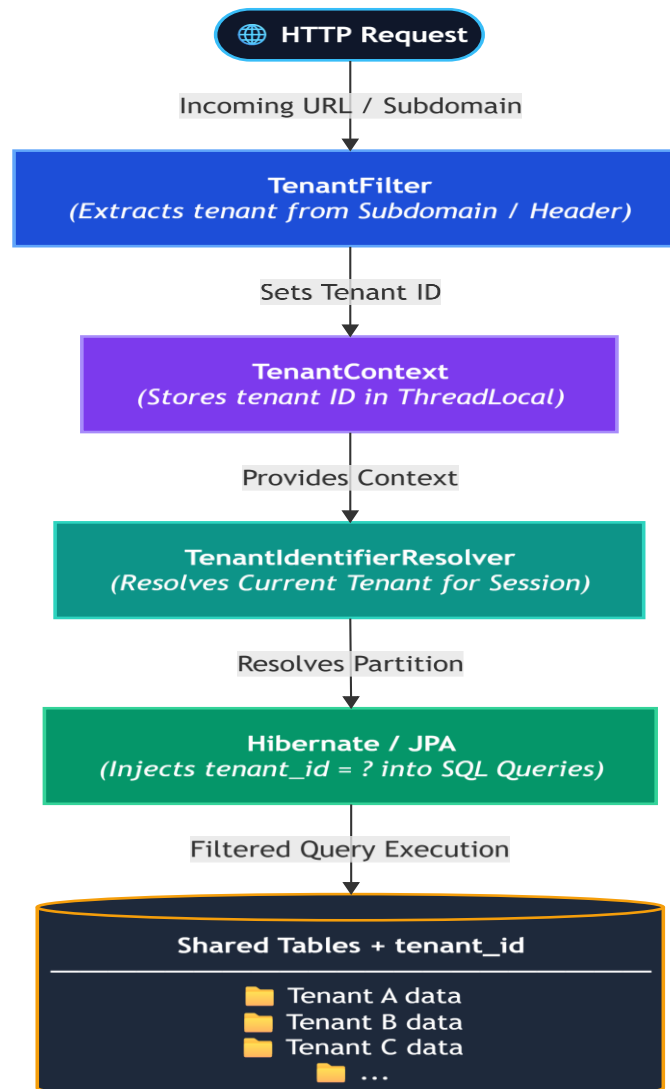


Fig 3.2: Tenant Isolation Structure

3.6.4 Behavioral Model

The behavioral model describes how VendorHood responds to requests during normal operation.

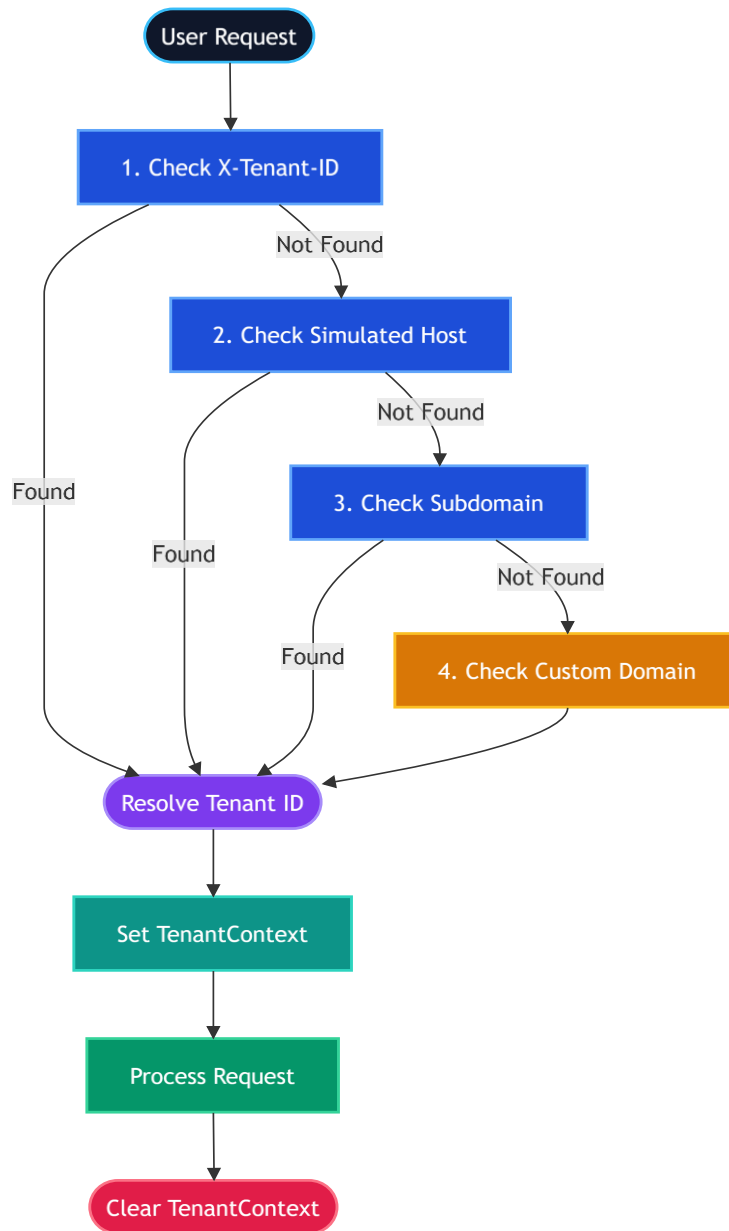


Fig 3.3: Behavioral Model

B. Order and Commission Processing

The order-processing behavior is:

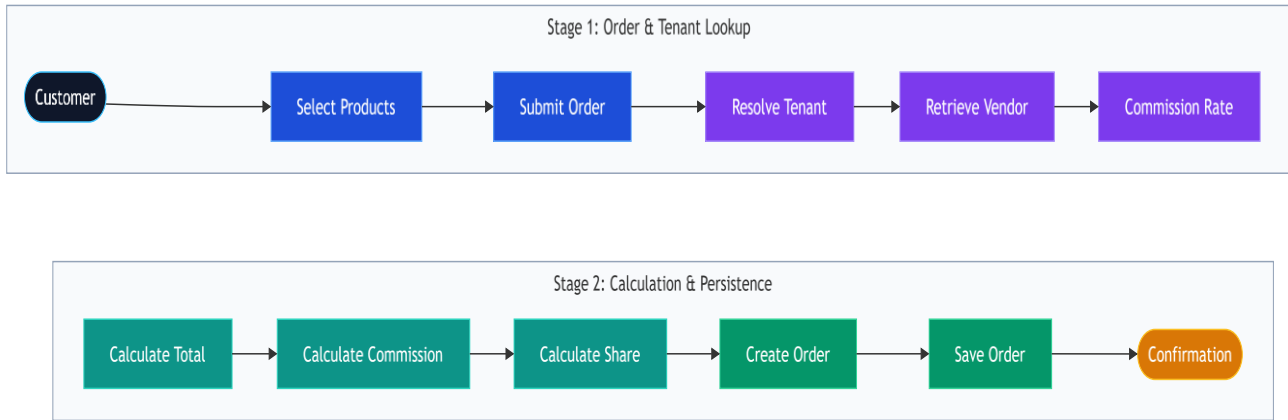


Fig 3.4: Order & Commision Processing

Mathematically:

$$C = P \frac{r}{100}$$
$$V = P - C$$

where C represents the platform commission and V represents the vendor share.

The current implementation performs these calculations in `StorefrontController` and stores the resulting values in the order record.

3.6.5 Storefront Template Behavior

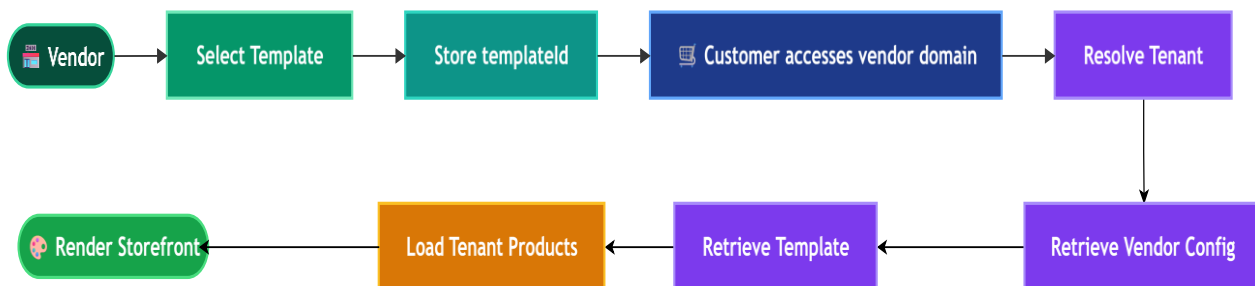


Fig 3.5: Storefront Template

The storefront rendering mechanism follows a configuration-based approach. The research paper describes this approach as a template-injection/front-end engine in which tenant-specific layout choices are stored as configuration rather than requiring separate storefront implementations. The frontend contains five predefined layout configurations that can be selected by vendors.

Chapter 4 - System Design and Experimental Set up

VendorHood follows a three-tier web architecture built around a single shared codebase that serves every tenant (vendor) from one running instance, rather than one deployment per client. The presentation tier is a browser-rendered React single-page application compiled into three role-specific consoles — a Customer Storefront, a Vendor Portal and a Platform Admin Console — assembled from the same set of components. Every request from these consoles reaches the application tier as a plain HTTPS/JSON call carrying an Authorization: Bearer token and, where relevant, an X-Tenant-ID header. On the server, a Spring Boot application exposes REST controllers (AuthController, AdminController, VendorController, StorefrontController) that sit behind a single custom TenantFilter. This filter decodes the caller's token, enforces role-based route guards (PLATFORM_ADMIN for /api/admin/**, VENDOR_ADMIN for /api/vendor/**), and resolves the active tenant either from the token, from an explicit X-Tenant-ID header, or — for public storefront traffic — from the request's Host header, by matching a subdomain such as kicks.vendorhood.com against the Vendor table. The resolved tenant identifier is placed on a ThreadLocal (TenantContext) for the lifetime of the request and consumed by a Hibernate CurrentTenantIdentifierResolver, which, combined with the @TenantId annotation on the Product and Order entities, causes Hibernate to silently append a tenant_id predicate to every query issued through the ORM. Physically, every tenant shares one MySQL 8 database and one set of tables — a shared-database, shared-schema, discriminator-column model — which keeps the deployment footprint of the platform to a single application server and a single database instance regardless of how many vendor storefronts it hosts. Figure 4.1 summarises this layout.

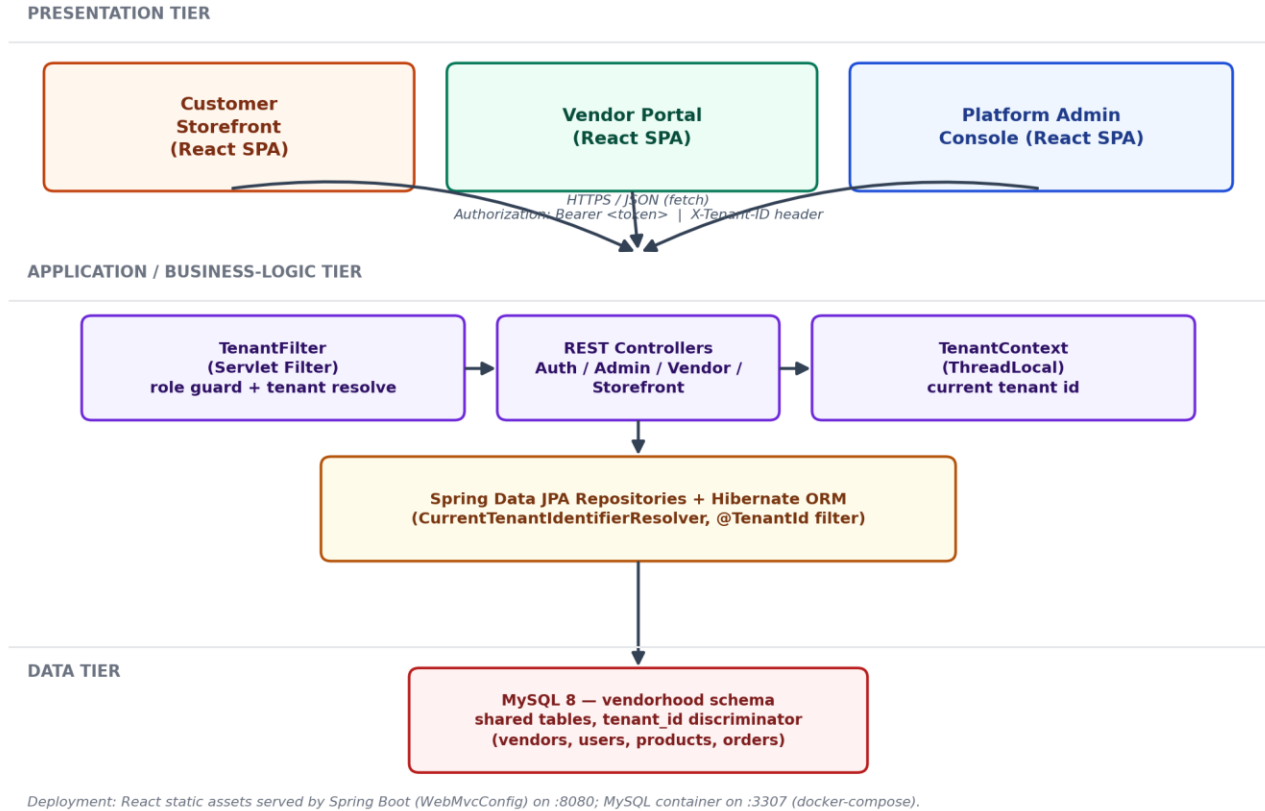


Fig. 4.1 — System architecture / physical layout block diagram of VendorHood.

4.2 Algorithm & Process Flow Design

Two process flows govern most of the platform's runtime behaviour: how a request is authenticated and routed to the correct tenant, and how a customer order is priced and split between the vendor and the platform. Both are implemented as straight-line procedural logic inside the Spring Boot backend rather than as external workflow engines, which keeps the request path short and easy to trace end to end.

4.2.1 Authentication and Tenant-Resolution Flow

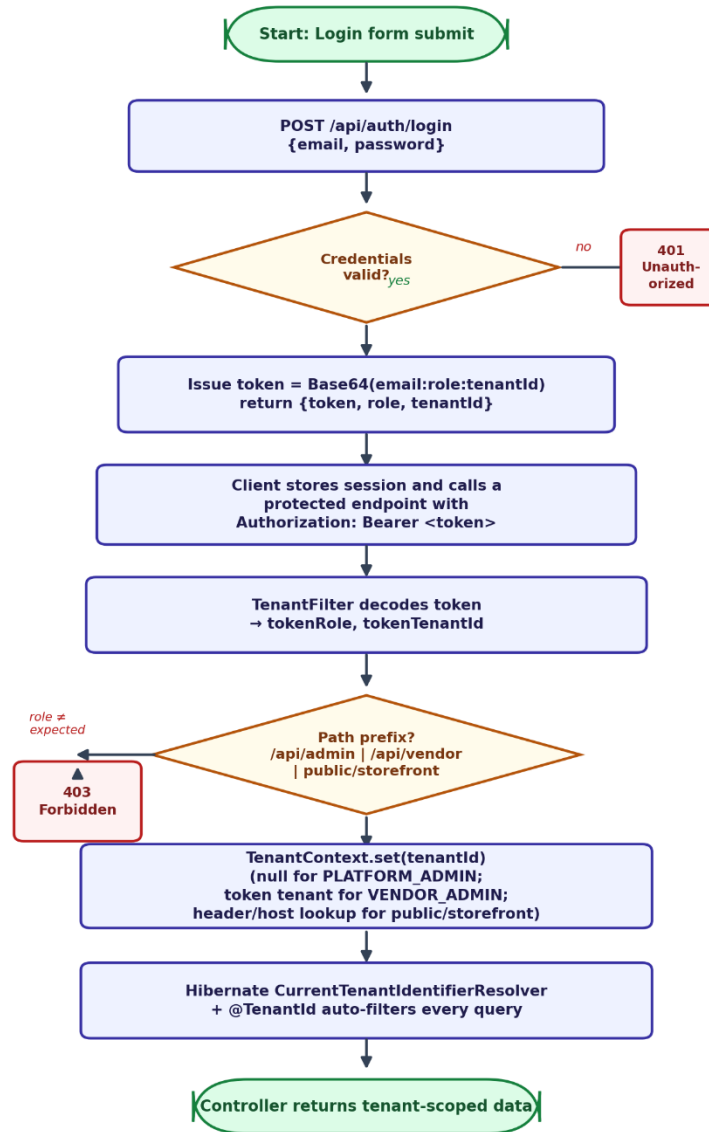


Fig. 4.2 — Flowchart: authentication and role-based tenant resolution.

Figure 4.2 traces a request from login through to a tenant-scoped response. `AuthController.login()` looks the submitted e-mail up in the users table and compares the stored password; on success it issues a lightweight bearer token built as `Base64(email:role:tenantId)` rather than a signed JWT. On every subsequent call, `TenantFilter` Base64-decodes this token to recover the caller's role and tenant id, applies the matching route guard, and — depending on whether the path belongs to the admin, vendor, or public/storefront namespace — either clears the tenant context (platform admin, sees every tenant), pins it to the token's own tenant (vendor admin, sees only their own store), or resolves it from the X-Tenant-ID header / Host header (public storefront traffic)— isolation is enforced once, centrally, in the filter and the ORM layer.

4.2.2 Checkout and Commission-Split Flow

Figure 4.3 shows the algorithm `StorefrontController.createStorefrontOrder()` runs on every checkout. The customer's cart total and the target vendor's tenant id (taken from the resolved storefront) are POSTed to `/api/storefront/orders` with an X-Tenant-ID header identifying the vendor. The controller loads that `Vendor` row to read its per-tenant `commissionRate`, computes $\text{platformCommission} = \text{round}(\text{total} \times \text{rate} / 100, 2)$ and $\text{vendorShare} = \text{round}(\text{total} - \text{platformCommission}, 2)$, and persists a new `Order` row with status "Pending". Because `Order` also carries `@TenantId`, the row is stamped with the resolved tenant automatically before it reaches MySQL. The frontend then re-fetches orders and products so that the customer's order history and the vendor's Sales tab stay consistent with the new record.

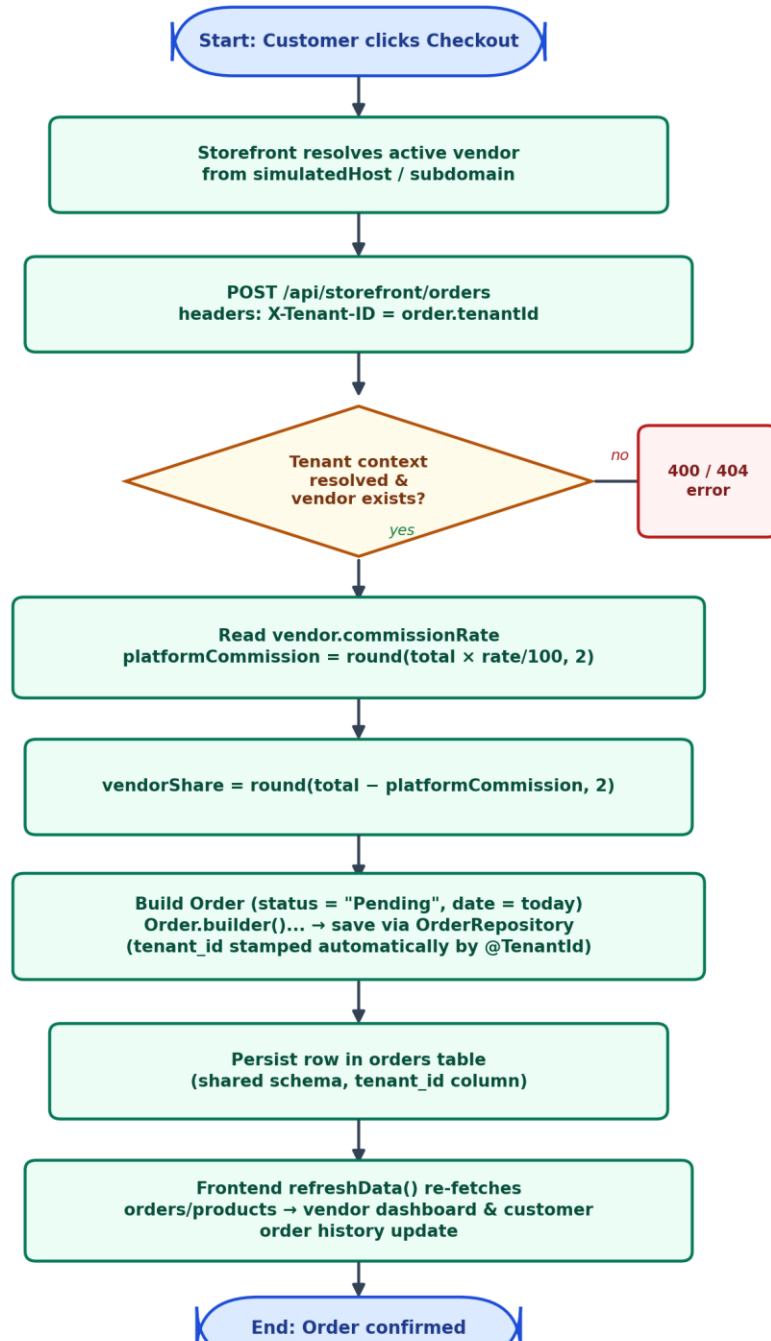


Fig. 4.3 — Flowchart: multi-tenant checkout and commission-split algorithm.

4.3 User Interface & Input Data Design

VendorHood ships four screens implemented as React components — LoginScreen.js, PlatformAdmin.js, VendorPortal.js and CustomerStorefront.js — all styled with Tailwind CSS utility classes and switched by a single currentView state machine held in App.js. Figure 4.4 lays out the structural regions of each screen as implemented in the component tree, annotated with the data each region reads or writes.

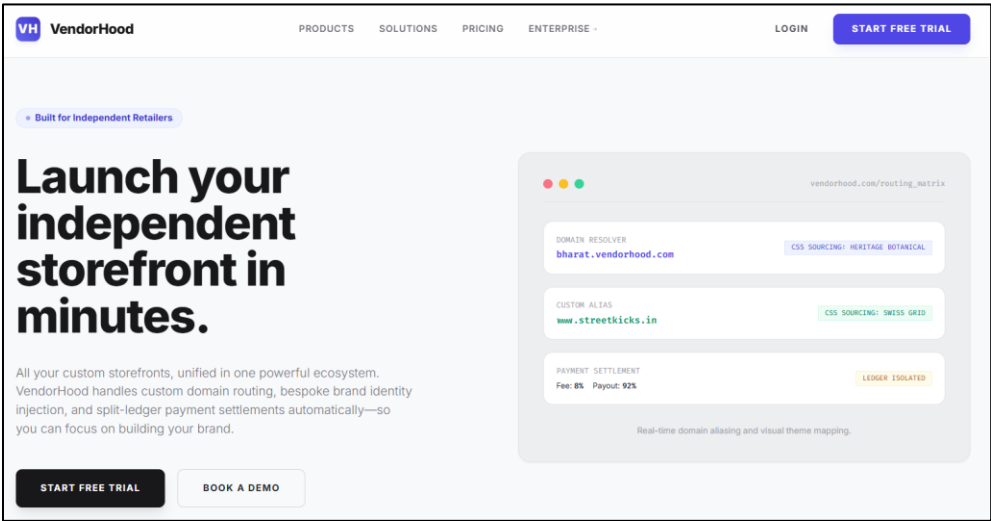
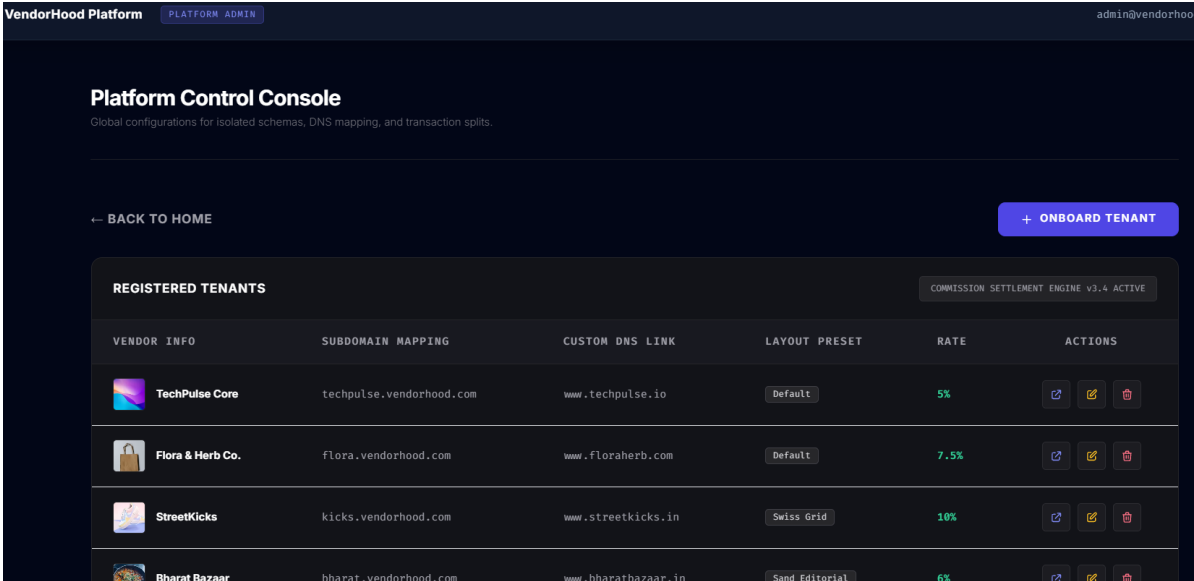


Fig. 4.4 — VendorHood Landing Page



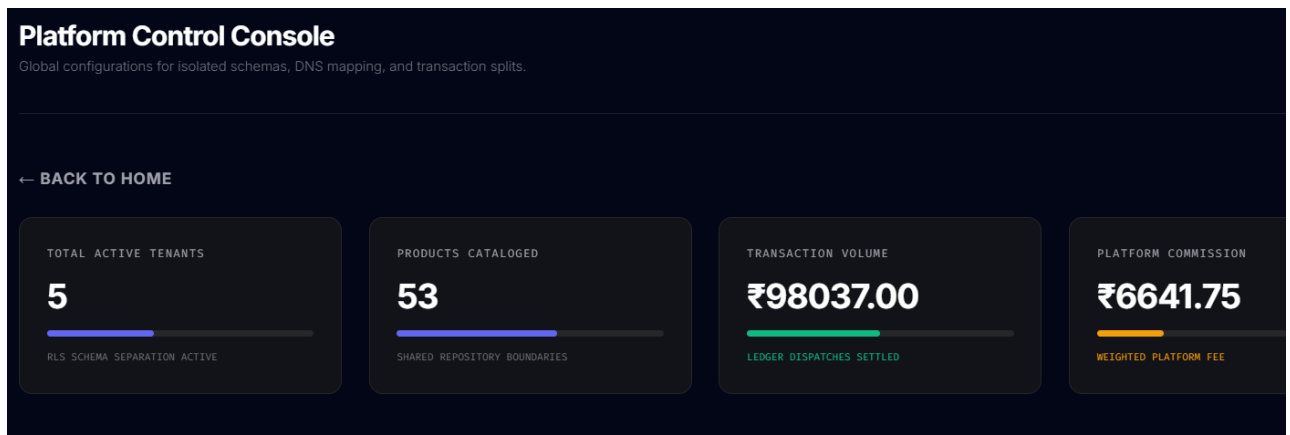


Fig 4.6 – Platform Control Console

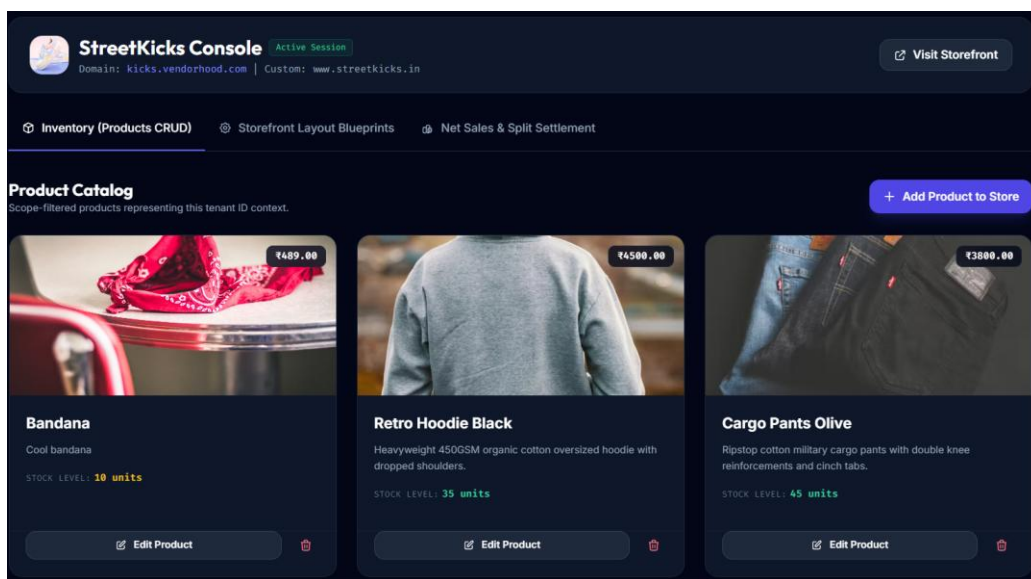


Fig. 4.7— Product Catalog/ Inventory for Vendors

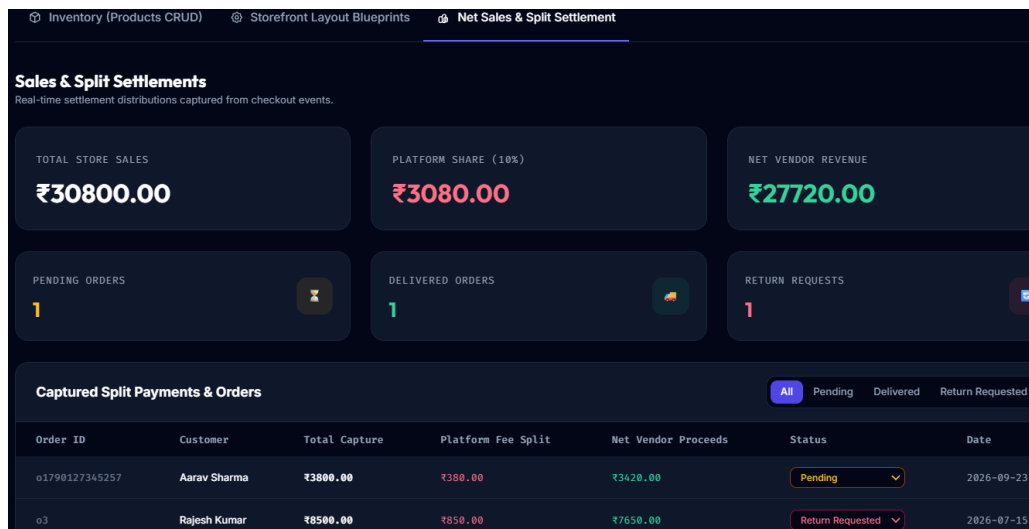


Fig. 4.8 — Shop Analytics+ Order Ledger for VendorHood Admins

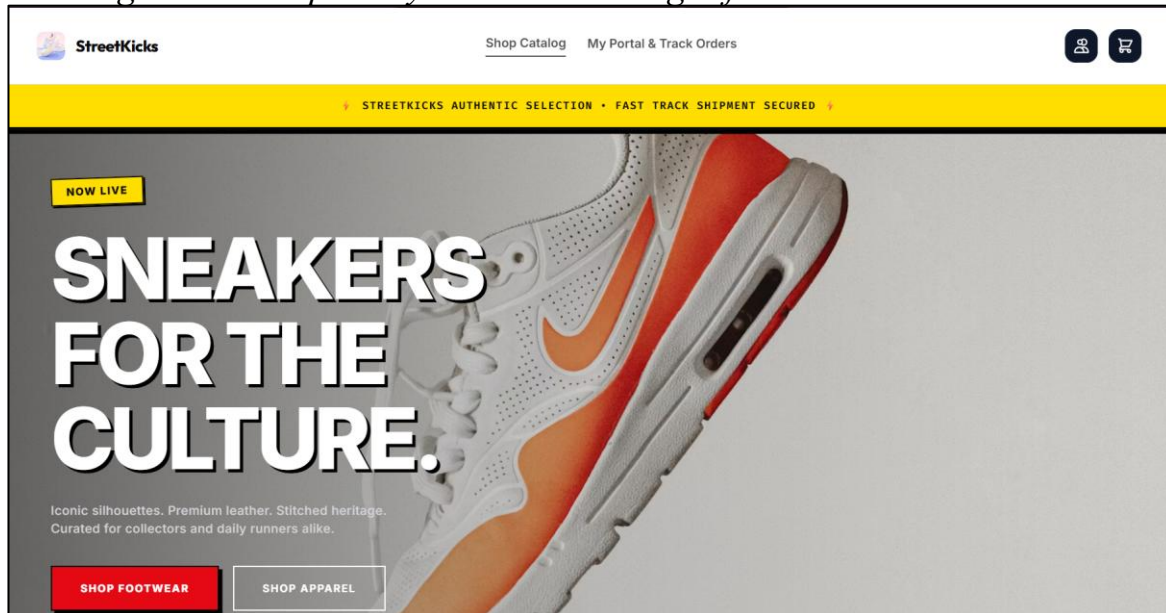


Fig. 4.9— Vendor Customizable Storefront

The Login screen is the single entry point for all three roles: it posts an e-mail/password pair to `/api/auth/login` and branches into one of the other three screens based on the role returned. The Platform Admin console exposes global KPIs (vendor count, gross sales, platform commission) sourced from AdminController's aggregate queries, a vendor-management table for onboarding new stores, and read-only cross-tenant products/orders tables. The Vendor Portal is tabbed into Inventory (per-product create/edit/delete forms), Styling (storefront template selection) and Sales (revenue/commission KPIs plus an order-status control). The Customer Storefront renders the active vendor's catalogue inside whichever of the five visual templates is assigned to that tenant, and drives a cart drawer and a checkout modal that captures the customer's name and e-mail before the order is submitted. Table 4.1 summarises the input fields captured on each screen.

Table 4.1 — Input data design summary

Screen / Form	Input fields	Type / notes
Login	Email, Password	text / password, required; verified server-side against the users table
Add / Edit Vendor (Admin)	Name, Subdomain, Custom domain, Template ID (1–5), Commission rate (%), Logo URL	text / number; subdomain and custom domain are enforced unique at the database level
Add / Edit Product (Vendor)	Name, Price, Description, Image URL, Stock, plus template-specific fields (hover image, size preview, trending / limited / streetwear flags, origin, weight, cuisine)	text / number / boolean; price and stock are required numeric fields
Vendor Profile	Store name, Template ID, Logo URL, Custom domain	text / number, scoped to the logged-in vendor's own tenant row
Checkout (Customer)	Customer name, E-mail, Cart line items (product id, quantity), Coupon code (optional)	text / email; cart subtotal computed client-side, commission split computed server-side
Order status update (Vendor)	Status — Pending / Delivered / Return Requested	enum selected from a dropdown, sent as a PUT request

4.4 Experimental Setup and Tools

Table 4.2 - Software Requirements

Layer	Technology	Purpose
Backend	Java 17, Spring Boot 3.3.2 (Spring Web, Spring Data JPA)	REST API layer, dependency injection, embedded application server
Backend	Hibernate ORM (@TenantId, CurrentTenantIdentifierResolver)	shared-schema multi-tenant persistence
Backend	MySQL Connector/J	JDBC driver for MySQL
Backend	Project Lombok	boilerplate reduction (@Data, @Builder, constructors)
Backend	Apache Maven	build and dependency management (pom.xml)
Database	MySQL 8.0 (containerised)	relational store — one "vendorhood" schema shared by all tenants
Frontend	React 18 (development UMD build, via CDN)	component-based UI for all three consoles
Frontend	Babel Standalone	in-browser JSX-to-JS transpilation, no build step

Layer	Technology	Purpose
Frontend	Tailwind CSS (Play CDN)	utility-first styling and per-tenant theme classes
Frontend	Google Fonts — Inter, Outfit, Playfair Display, Fira Code	typography for the five storefront templates
Tooling	Docker & Docker Compose	reproducible, disposable MySQL container
Tooling	Postman / browser DevTools	manual REST API testing
Tooling	Git	version control
IDE	IntelliJ IDEA / Visual Studio Code	backend and frontend development

Table 4.3 - Hardware Requirements

(a) Development / hosting machine — used to build and run the full stack (Spring Boot backend + MySQL container):

Component	Minimum	Recommended
Processor	Dual-core, 2 GHz	Quad-core, 2.5 GHz or better
RAM	4 GB	8 GB or more
Storage	~1 GB free (JDK, Maven cache, Docker image, DB data)	2 GB+ free on SSD
Operating system	Windows 10 / Ubuntu 20.04 / macOS 11 (64-bit)	any current 64-bit OS with Docker Desktop support
Required software	JDK 17, Docker Desktop, a modern web browser	same, plus an IDE (IntelliJ IDEA / VS Code)

(b) End-user machine — needed only to open and use the deployed website in a browser:

Table - 4.4 Implementation, Deployment and Testing

Component	Requirement
Device	Any desktop, laptop, tablet or smartphone capable of running a modern web browser
Processor / RAM	No requirement beyond what the browser itself needs (about 2 GB RAM is sufficient)
Browser	A current version of Chrome, Edge, Firefox or Safari with JavaScript enabled — the UI is a client-rendered React app compiled in-browser by Babel
Network	An active internet connection is required even after the page loads, to fetch React, Babel and Tailwind from their CDNs and the Google Fonts stylesheet, in addition to the app's own API calls

Component	Requirement
Display	Any screen size; the layout is responsive, though a width of 768 px or more gives the intended multi-column dashboard views
Installation	None — no plug-in or app download is needed to use the storefront, vendor, or admin consoles

4.5.1 Implementation

On the backend, the implementation follows a conventional Spring Boot layering of entity → repository → controller. The four JPA entities (User, Vendor, Product, Order) are written with Lombok's `@Data`, `@Builder`, `@NoArgsConstructor` and `@AllArgsConstructor` annotations to avoid hand-written getters, setters and constructors, and Product/Order additionally carry Hibernate's `@TenantId` on their `tenantId` column so that the shared-schema isolation described in Section 4.1 is enforced without any explicit filtering code in the repositories. Spring Data JPA interfaces (VendorRepository, ProductRepository, OrderRepository, UserRepository) supply the standard CRUD methods plus a small number of custom finder methods (`findBySubdomain`, `findByCustomDomain`, `findByEmail`) and, in ProductRepository / OrderRepository, native `@Query` methods (`findAllProductsGlobal`, `sumTotalAmountGlobal`, `sumPlatformCommissionGlobal`, and similar) that deliberately bypass the Hibernate tenant filter so that AdminController can compute genuinely cross-tenant statistics for the platform-wide dashboard. AdminController's vendor-deletion endpoint similarly falls back to raw `EntityManager.createNativeQuery()` calls to cascade-delete a vendor's products and orders, again because the entity-level `@TenantId` filter would otherwise hide rows belonging to any tenant other than the one currently in context.

The frontend was deliberately implemented without a Node build pipeline: `index.html` loads React 18, ReactDOM and Babel Standalone directly from CDN script tags, and each component file is included as a `<script type="text/babel">` tag that Babel transpiles in the browser at load time before `App.js` mounts the tree with `ReactDOM.createRoot()`. All cross-role UI logic — which console to show, which vendor is the "active tenant", and every CRUD call to the backend — is centralised in `App.js`, which holds `currentUser`, `currentView`, the `vendors/products/orders` arrays, and a `simulatedHost` value that stands in for a real wildcard-subdomain DNS setup during local development (`resolvedTenant` is derived from it with `useMemo` by matching against each vendor's subdomain or

customDomain). Each of the five storefront visual themes is defined once as a small object of Tailwind class strings inside a `STOREFRONT_TEMPLATES` map, and two tenants in the seed data (StreetKicks and Flora & Herb Co.) additionally render through purpose-built `SneakerStorefront.js` / `SpiceStorefront.js` components, demonstrating the platform's template-injection approach to per-tenant branding without maintaining a separate codebase per vendor.

4.5.2 Deployment

The reference deployment is intentionally lightweight and single-host. `docker-compose.yml` brings up one `mysql:8.0` container (`vendorhood-mysql`) with a persistent named volume, exposing the database on host port 3307 to avoid clashing with any locally-installed MySQL instance. The Spring Boot backend is built and run with Maven (`mvn spring-boot:run`, or as the packaged JAR produced by the `spring-boot-maven-plugin`) and listens on port 8080; on startup, Hibernate's `ddl-auto=update` setting creates or updates the `vendors` / `users` / `products` / `orders` tables directly from the entity classes. The static frontend needs no separate web server for local use — `WebMvcConfig` maps every unmatched path to the parent project directory (`file:../`), so the same Spring Boot process that serves the API can also serve `index.html`, `style.css` and the component scripts; alternatively, the front-end files can be hosted from any static file server or CDN with the API base URL adjusted accordingly. For a production rollout beyond the current prototype, the natural next steps are to move the database credentials out of `application.properties` and into environment variables or a secrets store, replace the Base64 pseudo-token with a signed, expiring JWT, restrict `WebMvcConfig`'s CORS policy from `allowedOriginPatterns("*")` to the platform's real domains, replace the `CDN-plus-in-browser-Babel` frontend with a proper production build for performance, and put a reverse proxy or wildcard DNS record in front of the application so that each vendor's subdomain or custom domain resolves to the same backend the way `TenantFilter.resolveTenantByHost()` already expects.

4.5.3 Testing

Testing at this stage of the project was carried out manually rather than through an automated suite, by walking each of the three role journeys end to end: a platform admin onboarding a new vendor and confirming it appears correctly in the cross-tenant dashboard

and statistics; a vendor admin adding, editing and deleting products, changing their storefront template, and moving an order through Pending → Delivered / Return Requested; and a customer browsing a specific vendor's storefront (by switching the simulated host / subdomain), adding items to the cart, applying a coupon, and completing checkout. Multi-tenant isolation was verified explicitly by logging into two different vendor accounts in turn and confirming that VendorController's product and order endpoints for one tenant never returned rows belonging to the other — the expected effect of the @TenantId / CurrentTenantIdentifierResolver pairing described in Section 4.1. Role-based access control was checked by calling admin-only and vendor-only endpoints with the wrong role's token and confirming that TenantFilter returned HTTP 403, and public endpoints were checked without a token to confirm that tenant resolution still worked from the X-Tenant-ID header and the Host-header fallback. API responses — status codes and JSON payload shape — for every controller method were also spot-checked with Postman and the browser's network inspector, and the CDN-based UI was exercised in the latest Chrome and Firefox to confirm the Tailwind-driven layouts render consistently. The spring-boot-starter-test dependency is already present in pom.xml, so JUnit 5 / Spring Test-based automated unit and integration tests — particularly around the tenant-filter logic and the commission-split arithmetic — are a planned addition beyond this manual test cycle.

Chapter 5 — Results & Discussion

5.1 Outputs & Outcomes

The proposed system results in a multi-tenant SaaS framework for online retail that enables multiple vendors to operate their individual storefronts using shared application infrastructure.

The major outputs and outcomes of the proposed system are:

1. Multi-Tenant Retail Platform

The framework allows multiple vendors to operate independently on a common platform while maintaining logical separation of their data.

2. Tenant-Based Data Isolation

Each product and tenant-specific record is associated with a Tenant ID. Tenant context is established for incoming requests and is used to restrict database operations to the corresponding tenant. This helps prevent unintended cross-tenant data access.

3. Customizable Storefront

The system provides a template-injection front-end engine. Vendors can select predefined storefront templates, store the corresponding page blueprint, and generate a tenant-specific storefront without developing an entirely separate frontend.

4. Domain-Based Tenant Identification

The framework supports tenant identification through both subdomains and custom domains. The incoming domain/Host header is used to determine the corresponding tenant.

5. Commission-Based Payment Processing

The proposed payment flow allows a transaction to be divided between the platform and vendor. The platform receives its commission while the remaining amount is allocated to the vendor.

6. Complete Retail Transaction Flow

The proposed workflow covers:

Vendor Registration → Tenant Creation → Tenant/Domain Assignment → Storefront Configuration → Customer Visit → Tenant Resolution → Login → Catalogue → Product Selection → Stock Verification → Checkout → Payment Split → Order Status Update → Order Confirmation.

Overall Outcome

The resulting framework demonstrates how shared infrastructure, tenant isolation, configurable storefronts, domain resolution, and transaction-based monetization can be combined into a single SaaS-oriented retail architecture.

5.2 Analysis of Results & Interpretation of Data

The results can be analysed based on the major architectural objectives of the proposed system.

Tenant Isolation

The shared-schema approach allows multiple tenants to use the same database structure while distinguishing their records through a tenant identifier. This provides a resource-efficient architecture, but correct tenant-context enforcement is essential because improper filtering could result in cross-tenant data exposure.

Interpretation:

The architecture demonstrates that shared infrastructure can be used for multiple vendors, provided that tenant identification and data-access restrictions are consistently enforced.

Storefront Customization

The template-based approach separates storefront structure from tenant-specific content. Instead of creating a separate application for each vendor, predefined page blueprints can be populated with tenant-specific information.

Interpretation:

This reduces the amount of custom development required for individual vendors while still allowing different tenants to maintain distinct storefront presentations.

Domain Resolution

The framework supports both platform subdomains and custom domains. The system determines the tenant based on the incoming domain and then establishes the corresponding tenant context.

Interpretation:

This allows individual vendors to have recognizable storefront addresses while continuing to operate on the shared SaaS infrastructure.

Payment Processing

The payment module separates the transaction into a platform commission and the remaining vendor amount.

Interpretation:

This supports the proposed transaction-based monetization model, where the platform's revenue is associated with successful retail transactions rather than requiring the vendor to bear the entire fixed cost of the platform.

Scalability and Performance

Because multiple tenants share application resources, the architecture can experience resource interference, particularly when one tenant generates significantly higher workloads than others. The paper identifies this as the noisy-neighbor problem and discusses resource control and more isolated deployment approaches as possible solutions.

Interpretation:

Multi-tenancy improves resource sharing and cost efficiency, but increasing workload may require additional resource-management or isolation mechanisms.

5.3 Discussion of Results & Limitations of the System

Discussion of Results

The proposed framework addresses the main problem identified in the research: the high fixed cost of establishing and operating independent online retail systems for small and micro-scale businesses. The combination of shared infrastructure, shared-schema multi-tenancy, tenant-level data isolation, template-based storefronts, subdomain/custom-domain resolution, and commission-based payment settlement provides an integrated architecture for SaaS-based online retail. The framework also demonstrates an important trade-off: while sharing resources can reduce infrastructure costs, it increases the importance of tenant isolation, security, resource management, and performance control. Therefore, the paper also considers alternative deployment approaches, including multi-tenant, multi-instance, and hybrid/microservice-oriented architectures.

Limitations of the System

1. Shared Resource Interference

Multiple tenants share common infrastructure. A tenant generating unusually high traffic or workload may consume more CPU, memory, or other resources and affect other tenants. This is referred to as the noisy-neighbour problem.

2. Complexity of Tenant Isolation

Because tenants use a shared database schema, incorrect tenant filtering or context handling could potentially expose data belonging to another tenant.

Therefore, tenant-context enforcement must be consistently applied throughout the application and data-access layers.

3. Limited Storefront Customization

The proposed template-injection approach relies on a predefined set of templates/page blueprints. Therefore, customization is more controlled than developing a completely independent storefront for every vendor.

4. Logistics Are Outside the Platform

The proposed system does not take responsibility for physical fulfilment and logistics. Vendors remain responsible for shipping and fulfilment activities.

5. Performance and Scalability Challenges

As the number of tenants and their workloads increase, resource contention can become more significant. The paper therefore identifies resource isolation and microservice-level tenancy as areas requiring further consideration.

6. Payment Dependency

The transaction settlement process depends on the payment module/gateway and its callbacks or webhooks for updating transaction and order status.

Chapter 6 — Conclusion & Future Scope

6.1 Conclusion

This paper presents a multi-tenant SaaS framework for online retail systems based on the principle that affordability and usability for small sellers are strongly influenced by how costs and risks are distributed over time. The literature reviewed in the paper identifies conventional fixed-cost requirements such as software development, customization, infrastructure setup, maintenance, and continuous operational expenses as significant barriers for businesses with limited capital. A multi-tenant approach addresses this cost structure by allowing multiple vendors to share common code and infrastructure while maintaining logical separation of their data.

The proposed framework applies this multi-tenant approach to an online retail environment by combining several architectural components into a single system. The framework uses shared-schema multi-tenancy with enforced tenant context to maintain logical separation between tenants and reduce the possibility of cross-tenant data access. Tenant-specific data is associated with a tenant identifier, while tenant context is established before business logic and data-access operations are performed. This provides a structured approach to maintaining data isolation within the shared environment.

The framework also incorporates a template-injection front-end engine to allow vendors to create customized storefronts without requiring independent development for every tenant. A tenant can select from predefined page designs, with the selected layout and component configuration stored as tenant-specific data. During runtime, the rendering engine uses the selected schema together with the tenant's content and catalogue to generate the corresponding storefront. This supports storefront customization while retaining a common rendering mechanism across the platform.

Another important component is domain-based tenant resolution, which supports both subdomain-based and custom-domain storefronts. Incoming requests can be examined through the Host header, allowing the system to identify a tenant either through a platform-defined subdomain or through a registered custom-domain alias. Once the tenant is resolved, the tenant identifier is bound to the request context and used to restrict subsequent data operations to the appropriate tenant. This integrates domain resolution with tenant identification and data isolation rather than treating them as independent functions.

The framework further introduces transaction-dependent monetization through commission-based payment settlement. Instead of depending entirely on fixed charges, the platform can take a configurable percentage of a transaction at payment time. The payment process divides the transaction between the vendor and the platform, with the platform receiving its commission and the vendor receiving the remaining amount. The settlement

is also associated with order-status updates through payment callbacks, connecting payment processing with the subsequent order flow.

The framework also recognizes that sharing infrastructure introduces important considerations related to performance isolation and resource management. In a shared Java environment, tenants may compete for CPU and memory resources, resulting in cross-tenant performance interference. The paper therefore considers the importance of runtime observability and protection against the noisy-neighbor effect. It also discusses fine-grained resource control and the possibility of applying tenancy management at the service level when the framework is divided into microservices.

The comparison of deployment approaches further considers multi-tenant, multi-instance, and hybrid microservice-based multi-tenant architectures in terms of operational cost, resource sharing, number of instances, privacy, isolation, and application design. The hybrid approach combines characteristics of shared and dedicated models, while also introducing greater architectural complexity. These alternatives demonstrate that deployment decisions involve trade-offs between resource sharing, isolation, privacy, cost, and architectural complexity.

Overall, the proposed framework provides an integrated architecture for SaaS-based online retail by combining shared infrastructure, tenant isolation, customizable storefronts, domain-based tenant resolution, commission-based settlement, and considerations for scalability and performance. The framework deliberately keeps logistics outside its scope, leaving order shipment and physical fulfilment to individual vendors. Instead, it concentrates on the programmable core of the retail platform, including identity, storefront creation, transaction processing, tenant management, and scaling while preserving tenant isolation.

6.2 Future Scope

The future scope of the proposed framework is closely related to the increasing role of automation and analytics in multi-tenant platforms. The literature indicates that tenant-facing customization is likely to become more automated, while analytic intelligence can become an important capability as unified platforms collect and process data across different activities. The proposed framework provides a foundation for such extensions while retaining its common rendering engine and multi-tenant architecture.

One important direction is the use of automated storefront synthetic AI agents as a continuation of the existing template-selection process. Multi-tenant builder systems can already be conceptualized around schema generation and rendering. Based on this approach, a generative agent may generate a valid page schema using natural-language constraints and populate component structures and content without violating the design system of the platform. In the context of retail tenants, these agents could act as an abstraction over the existing template-injection mechanism, reducing the cognitive effort

involved in storefront creation while maintaining the common rendering engine of the platform.

Another area of future scope is analytics and inventory forecasting intelligence. Once the platform gathers event streams such as page views, product interactions, shopping-cart activity, and purchases, these data can support analytics and recommendation logic. Earlier multi-tenant retail work has already considered extensions involving the analysis of viewed and purchased products, while broader multi-tenant platform descriptions refer to cross-module analytics and predictive forecasting enabled by integrated data. For the proposed framework, such analytics would be oriented toward actionable vendor requirements, including predicting stockouts, identifying high-velocity SKUs, and generating demand signals that can be

Appendix A

Abbreviation and Symbols

SaaS: Software as a Service

API: Application Programming Interface

REST: Representational State Transfer

JPA: Java Persistence API

ORM: Object-Relational Mapping

DFD: Data Flow Diagram

UML: Unified Modeling Language

Hibernate: Object-Relational Mapping framework used for Java persistence

Tenant ID: Identifier used to associate data with a specific tenant

JWT: JSON Web Token

Definitions

Multi-Tenancy: An architecture in which multiple independent tenants use a shared application or infrastructure while their data remains logically separated.

SaaS: A software delivery model in which users access an application as a service instead of maintaining a separate software installation.

Tenant: An independent vendor or customer organization that uses the shared SaaS platform.

Tenant Isolation: The mechanism that prevents one tenant from accessing another tenant's data or configuration.

Shared Schema: A database design in which multiple tenants use the same database tables, with a tenant identifier distinguishing their records.

Noisy Neighbor: A condition in a shared environment where one tenant's heavy resource usage can affect the performance experienced by other tenants.

Storefront: The tenant-specific online retail interface through which customers browse products and place orders.

Commission: A percentage of a transaction retained by the platform, with the remaining amount allocated to the vendor.

Appendix C

Publications

- [1] F. Shaikh and D. Patil, "SaaS based Multi-tenant E-commerce to reduce cost and improve resource utilization," 2015 IEEE 9th International Conference on Intelligent Systems and Control (ISCO), Coimbatore, India, 2015, pp. 1-6, doi: 10.1109/ISCO.2015.7282302.
- [2] Pushpan, Shinu. (2024). Multi-Tenant Architecture: A Comprehensive Framework for Building Scalable SaaS Applications. International Journal of Scientific Research in Computer Science, Engineering and Information Technology. 10. 1117-1126. 10.32628/CSEIT241061151.
- [3] Rathod, Rohit & Londhe, Omkar & Waghmare, Harshad & Varande, Ajay & Khedikar, Sarita. (2025). A Multi-Vendor SaaS website builder with Role-Based CRM. K. Elissa, "Title of paper if known," unpublished.
- [4] Kumar, Ritesh. (2020). Multi-Tenant SaaS Architectures: Design Principles and Security Considerations. 5. 49-62. 10.35629/8193-05024962.
- [5] Sumit Kalra and T. V. Prabhakar. 2018. Towards Dynamic Tenant Management for Microservice based Multi-Tenant SaaS Applications. In Proceedings of the 11th Innovations in Software Engineering Conference (ISEC '18). Association for Computing Machinery, New York, NY, USA, Article 12, 1–5. doi: 10.1145/3172871.3172882M.
- [6] Z. Zhuang, C. Tran, H. Ramachandra and B. Sridharan, "Ensuring High-Performance of Mission-Critical Java Applications in Multi-tenant Cloud Platforms," 2014 IEEE 7th International Conference on Cloud Computing, Anchorage, AK, USA, 2014, pp. 618-626, doi: 10.1109/CLOUD.2014.88.
- [7] F. Shaikh and D. Patil, "Multi-tenant e-commerce based on SaaS model to minimize IT cost," 2014 International Conference on Advances in Engineering & Technology Research (ICAETR - 2014), Unnao, India, 2014, pp. 1-4, doi: 10.1109/ICAETR.2014.7012861.
- [8] C. Bemagoni and S. B. Kare, "Cloud Based E-Commerce Platform for Multi-Tenant and Multi-Vendor," International Journal of Engineering Research & Technology (IJERT), vol. 3, no. 9, pp. 96–99, Sept. 2014, Art. no. IJERTV3IS090134.

- [9] Pinto, Hugo & Luz, J & Ramos de Oliveira, Ricardo & Lopes de Souza, Paulo & Souza, Simone. (2016). A Systematic Mapping Study on the Multi-tenant Architecture of SaaS Systems. 10.18293/SEKE2016-068.
- [10] Makki, Majid & Landuyt, Dimitri & Lagaisse, Bert & Joosen, Wouter. (2021). Thread-level resource consumption control of tenant custom code in a shared JVM for multi-tenant SaaS. Future Generation Computer Systems. 115. 351-364. 10.1016/j.future.2020.09.025.
- [11] Ochei, Laud & Petrovski, Andrei. (2018). A Decision-based Model for Optimizing the Deployment of Cloud-hosted Application Components to Guarantee Multitenancy Isolation. International Journal of Intelligent Computing Research. 9. 890-901. 10.20533/ijicr.2042.4655.2018.0108.
- [12] Anusuru, Anil. (2021). Transforming E-Commerce Platforms through Multi-Tenant Architectures and Microservices. International Journal of Communication Networks and Information Security. 13.